

UrbanElectra

Professor Viva — Complete Q&A Preparation Guide

Read every answer out loud at least 3 times before your viva

Covers: Architecture · Auth · Database · Stripe · Redis · Orders · Admin · Security · Why Decisions

1. General & Project Overview

These are always the first questions. Answer confidently — this is your project, own it.

[EASY] Explain your project in one minute.

✓SAY THIS:

UrbanElectra is a full-stack e-commerce platform for electronics built on the MERN stack — MongoDB, Express, React, and Node.js. Customers can browse products by category, add them to a cart, apply discount coupons, enter a shipping address, and pay using Stripe. After payment, an order is created and tracked through statuses like pending, shipped, and delivered. Customers can cancel, return, or exchange orders. There is also an admin dashboard where admins can manage products, handle orders, process refunds, and view sales analytics. It is deployed live on Vercel.

🔗 **PRO TIP:** *Don't rush this. Speak slowly. Professors respect students who can summarise their own work clearly.*

[EASY] Why did you choose the MERN stack?

✓SAY THIS:

I chose MERN because all four technologies use JavaScript, so there is no context switching between languages. MongoDB's document model suits e-commerce data well because orders embed product snapshots naturally. React's component model makes complex UIs like the cart and admin dashboard manageable. Node and Express are lightweight and fast for building REST APIs. The entire ecosystem has strong community support and is widely used in industry.

🔗 **PRO TIP:** *Add: 'Also, MERN is very relevant for internships and jobs right now, so I wanted to get hands-on experience with it.'*

[MEDIUM] What is the overall architecture of your application?

✓SAY THIS:

The application is split into three layers. The frontend is a React single-page application built with Vite. It communicates with the backend via REST API calls using Axios. The backend is an Express server running on Node.js. It handles routing, authentication middleware, and business logic through controllers. The data layer uses MongoDB Atlas for persistent storage, Upstash Redis for caching and session management, Cloudinary for image storage, and Stripe for payments. In production on Vercel, the React build is served as static files and the Express backend runs as a serverless function.

🔗 **PRO TIP:** *Draw a quick diagram on paper while explaining this — it shows confidence.*

[MEDIUM] What is a REST API and how did you use it here?

✓SAY THIS:

A REST API is an architectural style for building web services where each URL represents a resource and HTTP methods define the action. GET retrieves data, POST creates, PUT or PATCH updates, and DELETE removes. In my project, for example, GET /api/products/featured returns featured products, POST /api/cart adds an item to the cart, and DELETE /api/cart removes one. Each endpoint is stateless — the server does not remember the previous request, all needed information is sent with each request.

🔗 **PRO TIP:** *easy*

2. Authentication & JWT

Auth is the most commonly questioned topic in vivas. Understand every step cold.

[MEDIUM] What is JWT and how does it work?

✓SAY THIS:

JWT stands for JSON Web Token. It is a compact, URL-safe token that encodes a JSON payload and signs it with a secret key. When a user logs in, the server creates a JWT containing the user's ID, signs it with a secret, and sends it to the client. On every subsequent request, the client sends this token back. The server verifies the signature — if it is valid, the server trusts the identity without checking the database. A JWT has three parts separated by dots: the header, which specifies the algorithm; the payload, which holds claims like `userId`; and the signature, which prevents tampering.

🔗 **PRO TIP:** *If asked 'can you decode a JWT?' — yes, the payload is just Base64 encoded, not encrypted. The signature is what makes it secure.*

[HARD] Why do you have two tokens — access token and refresh token?

✓SAY THIS:

The access token is short-lived — it expires in 15 minutes. This limits the damage if it is stolen. But asking the user to log in every 15 minutes is terrible UX. So we also issue a refresh token that lasts 7 days. When the access token expires, the client automatically calls `/api/auth/refresh-token` with the refresh token. The server validates it and issues a new access token silently, without the user noticing. This gives us the security of short-lived tokens and the convenience of staying logged in.

🔗 **PRO TIP:** *Professor will love this answer. This is a real industry pattern called 'silent refresh'.*

[HARD] Why are tokens stored in HttpOnly cookies instead of localStorage?

✓SAY THIS:

If tokens are stored in `localStorage`, any JavaScript on the page can read them. This means an XSS attack — where malicious script is injected into the page — can steal the token and hijack the session. `HttpOnly` cookies cannot be accessed by JavaScript at all. The browser sends them automatically with every request but they are invisible to scripts. This makes them much more secure against XSS attacks.

🔗 **PRO TIP:** *If asked about CSRF: 'I set `sameSite: strict` on the cookies, which means the browser will not send them on cross-site requests, preventing CSRF attacks.'*

[HARD] Why do you store the refresh token in Redis?

✓SAY THIS:

JWTs are stateless — once issued, the server cannot invalidate them before they expire. This is a problem for logout: if someone steals your refresh token, you cannot block it. By storing the refresh token in Redis and checking it on every refresh request, we make tokens stateful just for the refresh step. When a user logs out, I delete the Redis key. Now even if someone has the old refresh token, the server will reject it because the key no longer exists in Redis.

🔗 **PRO TIP:** *This is the most impressive part of your auth system. Practise this explanation until it is automatic.*

[MEDIUM] What happens when the access token expires? Walk me through it.

✓SAY THIS:

When the access token expires, the next API call returns a 401 Unauthorized response. The Axios interceptor in the frontend catches this 401. It calls `POST /api/auth/refresh-token`, sending the refresh token cookie. The server decodes the refresh token, looks up the stored token in Redis, and compares them. If they match, it generates a new access token and sets it as a cookie. Then the interceptor automatically retries the original failed request, which now succeeds. The user sees nothing — it is completely seamless.

🔗 **PRO TIP:** *hard*

[MEDIUM] What is bcrypt and why is it used?

✓SAY THIS:

bcrypt is a password hashing function. Passwords must never be stored as plain text in the database. bcrypt transforms the password into a fixed-length hash using a salt — a random value added before hashing to prevent two identical passwords from producing the same hash. The work factor, which I set to 10, controls how computationally expensive the hashing is, making brute-force attacks slow. When a user logs in, I use `bcrypt.compare()` to check the entered password against the stored hash.

 **PRO TIP:** *easy*

3. Database & MongoDB

Know your models inside out. The professor may ask you to draw them or explain every field.

[MEDIUM] Why did you choose MongoDB over a relational database like MySQL?

✓SAY THIS:

E-commerce data has variable structure. An order embeds a snapshot of product prices at the time of purchase — this is natural in a document database. MongoDB's flexible schema also allowed me to add fields like `statusHistory`, `trackingNumber`, and `returnReason` to the `Order` model without needing migrations. For this project, the relationships are not deeply relational, so the overhead of joins in SQL was not justified. MongoDB Atlas also offered free cloud hosting which suited the project.

🔗 **PRO TIP:** *If pushed: 'For a banking system with complex transactions I would use PostgreSQL. MongoDB fit this use case well.'*

[MEDIUM] Explain your User model.

✓SAY THIS:

The `User` model stores name, email as a unique lowercase trimmed string, password as a bcrypt hash, `cartItems` as an array of objects each containing a product reference and a quantity, and a `role` field that is either customer or admin. There is a pre-save hook that automatically hashes the password whenever it is modified. There is also a `comparePassword` method that uses `bcrypt.compare` to verify a plain password against the hash during login.

🔗 **PRO TIP:** *medium*

[HARD] Explain your Order model and why it is designed that way.

✓SAY THIS:

The `Order` model stores a reference to the `User`, an array of products where each entry has a product reference, a quantity, and the price at time of purchase. Storing the price in the order is intentional — product prices can change later, but the order must always reflect what the customer actually paid. It also stores `totalAmount`, the Stripe session ID as a unique field, a `shippingAddress` sub-document, a `status` enum with 11 possible states, a `statusHistory` array that logs every status change with a timestamp and note, tracking information, and refund details.

🔗 **PRO TIP:** *The price snapshot design decision is very likely to be asked about separately.*

[HARD] Why do you store the price inside the order's products array, not just a reference to the product?

✓SAY THIS:

Because product prices can change over time. If I only stored a product reference, and the admin later updated the price, the old order would show the new price — which is wrong. By copying the price into the order at checkout time, I preserve what the customer actually paid. This is called a price snapshot and is standard practice in e-commerce systems.

🔗 **PRO TIP:** *This is a design decision question. Answer it with confidence — it shows you thought about data integrity.*

[MEDIUM] What is the statusHistory array in the Order model for?

✓SAY THIS:

It is an audit trail of every status change the order goes through. Every time the order status changes — whether by the customer cancelling or the admin updating it — I push a new entry into `statusHistory` containing the new status, a timestamp, and a note. This means you can see the complete timeline of an order: when it was placed, when it was confirmed, when it shipped, and so on. I also use it in the return logic — I find the delivered entry in `statusHistory` to calculate whether the 30-day return window has expired.

🔗 **PRO TIP:** *hard*

[MEDIUM] What is Mongoose and what is a schema?

✓SAY THIS:

Mongoose is an ODM — Object Document Mapper — for MongoDB and Node.js. It lets you define schemas, which are blueprints that describe the structure and data types of documents in a collection. Without

Mongoose you get raw JSON in and out of MongoDB with no validation. With Mongoose, you define that price must be a Number and cannot be negative, that email must be unique, and so on. Mongoose enforces these rules before saving to the database and also provides helper methods like `findById` and pre-save hooks.

🔗 **PRO TIP:** *easy*

4. Stripe & Payments

Walk through the payment flow in order. Professors love asking 'what happens step by step'.

[HARD] Explain the complete payment flow from cart to order creation.

✓SAY THIS:

When the user clicks Proceed to Checkout, the frontend sends a POST request to `/api/payments/create-checkout-session` with the cart products, an optional coupon code, and the shipping address. The server builds a list of line items, applies any coupon discount, serialises the products and shipping address into the Stripe session metadata, and calls the Stripe API to create a checkout session. The server returns the session URL. The browser redirects the user to Stripe's hosted payment page. After payment, Stripe redirects to `/purchase-success` with a `session_id` in the URL. The frontend calls `POST /api/payments/checkout-success` with that session ID. The server retrieves the session from Stripe, verifies the payment status is 'paid', parses the products from metadata, creates the Order document in MongoDB, and deactivates the coupon if one was used.

PRO TIP: Say 'Stripe's hosted payment page' — this shows you know Stripe handles the card details, not you.

[HARD] Why do you store products in Stripe session metadata?

✓SAY THIS:

Stripe's checkout session is created on the server before the user pays. After payment, Stripe redirects back to my server, but at that point I need to know what was ordered to create the Order document. The session metadata is a key-value store that Stripe attaches to the session and returns when I retrieve it. I serialise the product IDs, quantities, and prices as JSON into the metadata. When checkout succeeds, I parse it back out. The shipping address is also stored there the same way.

PRO TIP: If asked about the 500 character metadata limit: 'I truncate shippingAddress to 490 characters as a safety measure because Stripe metadata values have a 500 character limit.'

[HARD] What prevents a duplicate order if the user refreshes the success page?

✓SAY THIS:

The `stripeSessionId` field on the Order model has a unique index in MongoDB. Before creating a new order, I check if an order with that session ID already exists. If it does, I return the existing order ID without creating a duplicate. This makes the checkout-success endpoint idempotent — calling it multiple times with the same session ID produces the same result safely.

PRO TIP: The word 'idempotent' will impress your professor. Use it.

[MEDIUM] How does the coupon system work?

✓SAY THIS:

Each user can have one active coupon at a time stored in the Coupon collection. When the user applies a coupon code, the server validates it belongs to that user, is active, and has not expired. If valid, the discount percentage is returned. During checkout, the discount is applied server-side before creating the Stripe session using a Stripe coupon object. After successful payment, the coupon's `isActive` field is set to false so it cannot be reused. A new loyalty coupon is automatically generated if the order subtotal was \$200 or more.

PRO TIP: *medium*

[MEDIUM] How do refunds work?

✓SAY THIS:

The admin clicks Process Refund on an order in the dashboard. The server retrieves the Stripe checkout session using the stored `stripeSessionId`, gets the `payment_intent` ID from it, and calls the Stripe Refunds API with that payment intent. The refund can be full or partial — the admin can enter a specific amount. Stripe processes the refund back to the customer's card. The server then stores the Stripe refund ID, the refund amount, the timestamp, and updates the order status to 'refunded'.

PRO TIP: *medium*

5. Redis & Caching

Redis is used for two distinct things in your project. Know both cold.

[MEDIUM] What is Redis and why did you use it?

✓SAY THIS:

Redis is an in-memory key-value data store. Because it stores data in RAM rather than on disk, reads and writes are extremely fast — typically under a millisecond. I used Redis for two purposes: first, to store refresh tokens for authentication, and second, to cache the list of featured products so that every homepage load does not need to query MongoDB.

🔗 **PRO TIP:** *easy*

[MEDIUM] How does Redis caching work for featured products?

✓SAY THIS:

When a request comes in for GET /api/products/featured, the server first checks if the key 'featured_products' exists in Redis. If it does, it returns the cached JSON immediately without touching MongoDB. If it does not exist — on the first request or after a cache miss — the server queries MongoDB, stores the result in Redis as a JSON string, and returns it. Whenever an admin toggles a product's featured status, the server immediately updates the Redis cache with the new list so it stays consistent.

🔗 **PRO TIP:** *If asked 'what if Redis is down?' — 'I catch Redis errors silently and fall back to MongoDB, so the site keeps working.'*

[MEDIUM] What is cache invalidation and how do you handle it?

✓SAY THIS:

Cache invalidation is the problem of keeping cached data in sync with the real data. In my project, the featured products cache can become stale if an admin changes which products are featured. I handle this with a write-through strategy: whenever toggleFeaturedProduct is called, I immediately overwrite the Redis cache with the fresh data from MongoDB. This way the cache is never stale for more than the duration of a single admin action.

🔗 **PRO TIP:** *hard*

[MEDIUM] Why store refresh tokens in Redis instead of MongoDB?

✓SAY THIS:

Two reasons. First, speed — token validation happens on every API call that needs a refresh, so it needs to be fast. Redis lookups are sub-millisecond. Second, TTL support — Redis has built-in key expiry. I set the refresh token to expire after 7 days automatically. With MongoDB I would need a separate cleanup job or a TTL index. Redis handles this natively with a single EX parameter.

🔗 **PRO TIP:** *hard*

6. React & Frontend

Focus on state management and component design — these are the common frontend questions.

[MEDIUM] What is Zustand and why did you use it instead of Redux?

✓SAY THIS:

Zustand is a lightweight state management library for React. It lets you create global stores — shared state that any component can read or update — without the boilerplate Redux requires. In Redux you need actions, action creators, reducers, and a provider. In Zustand you define state and update functions in one place and call them directly. For this project, I have four stores: `useUserStore` for authentication state, `useCartStore` for cart and totals, `useProductStore` for product listings, and `useOrderStore` for order management.

🔗 PRO TIP: [medium](#)

[MEDIUM] What is the difference between state and props in React?

✓SAY THIS:

Props are read-only inputs passed from a parent component to a child. They flow downward and the child cannot change them. State is data that a component owns and can change, causing a re-render when it changes. For example, in my `AddressForm` component, the form field values are local state managed with `useState`. The final confirmed address is lifted up to the Zustand cart store so the `OrderSummary` component — which is a sibling, not a parent — can also access it.

🔗 PRO TIP: [easy](#)

[MEDIUM] What is `useEffect` and when did you use it?

✓SAY THIS:

`useEffect` is a React hook that lets you run side effects after a component renders. Side effects are things like fetching data, setting up subscriptions, or updating the document title. I use it in many places — for example in `HomePage` I call `fetchFeaturedProducts` inside a `useEffect` so that when the page first loads, it fetches the featured products from the API. Without `useEffect`, the fetch would run on every single render, which would cause infinite loops.

🔗 PRO TIP: [easy](#)

[MEDIUM] What is Vite and why did you use it instead of Create React App?

✓SAY THIS:

Vite is a modern frontend build tool. It uses native ES modules in the browser during development, which means it only processes the files you are actually using rather than bundling the entire application. This makes the development server start almost instantly and hot module replacement — updating the browser when you save a file — is nearly instantaneous. Create React App uses Webpack which bundles everything upfront, making it much slower especially as the project grows.

🔗 PRO TIP: [medium](#)

[MEDIUM] What is Framer Motion and where did you use it?

✓SAY THIS:

Framer Motion is a React animation library. I used it to add entrance animations on page components — they fade in and slide up when the page loads — and micro-interactions on buttons, like a slight scale on hover and tap. It makes the UI feel more polished and professional. The key components are `motion.div` and `motion.button`, where you pass `initial`, `animate`, and `transition` props to describe the animation.

🔗 PRO TIP: [easy](#)

7. Order Lifecycle & Business Logic

The order state machine is one of the most impressive parts of your project. Know every status.

[MEDIUM] Walk me through all the order statuses.

✓SAY THIS:

An order starts as 'pending' when created after payment. The admin changes it to 'confirmed' when they acknowledge the order, then 'processing' while it is being packed. When the admin sets tracking information, the status automatically moves to 'shipped'. The admin then marks it 'delivered'. From delivered, the customer can request a 'return_requested' or 'exchange_requested'. The admin can then move those to 'returned' or 'exchanged'. At any point the admin can process a Stripe refund which sets the status to 'refunded'. Customers can cancel only from 'pending' or 'confirmed' — once an order is being processed it cannot be cancelled.

🔗 **PRO TIP:** Draw the state diagram on paper while explaining. It shows you truly understand it.

[MEDIUM] How does the 30-day return window work technically?

✓SAY THIS:

When a customer requests a return, I look through the statusHistory array in reverse to find the entry where status was 'delivered'. I get the changedAt timestamp from that entry and calculate the difference between now and that timestamp in days. If it is more than 30 days, I return a 400 error saying the return window has expired. This means the window is based on when the order was actually marked as delivered, not when it was placed.

🔗 **PRO TIP:** hard

[MEDIUM] What is the statusHistory used for and why not just store the current status?

✓SAY THIS:

Storing only the current status would lose all history. If a customer says 'my order was marked delivered but I never received it', without history I cannot see when it was marked delivered or by whom. The statusHistory array gives a complete audit trail — every status change is recorded with a timestamp and a note. This is essential for dispute resolution and also powers the order timeline UI that customers can see in their orders page.

🔗 **PRO TIP:** hard

8. Cloudinary & Image Handling

[MEDIUM] Why use Cloudinary instead of storing images on your server?

✓SAY THIS:

Storing images on the server has several problems. First, server disk space is limited and expensive. Second, on Vercel, serverless functions are stateless — any file you write to the filesystem is lost when the function terminates. Third, serving large images from the same server that handles API requests hurts performance. Cloudinary is a cloud-based image management service. I upload images to Cloudinary and store only the URL in MongoDB. Cloudinary serves images over a CDN, which is faster globally, and handles optimisation like compression and format conversion automatically.

🔗 **PRO TIP:** [medium](#)

[MEDIUM] How does image upload work in your project?

✓SAY THIS:

In the admin's CreateProductForm, when a file is selected, the FileReader API reads it and converts it to a base64-encoded string — a text representation of the binary image data. This base64 string is sent in the POST request body to /api/products. On the server, the Cloudinary SDK's uploader.upload() method accepts the base64 string, uploads it to the products folder in Cloudinary, and returns a secure_url. That URL is stored in the product's image field in MongoDB.

🔗 **PRO TIP:** [medium](#)

[MEDIUM] What happens to the Cloudinary image when a product is deleted?

✓SAY THIS:

When a product is deleted, I do not just delete the MongoDB document — I also delete the image from Cloudinary to avoid accumulating orphaned files and wasting storage. I parse the product's image URL to extract the public ID, which is the identifier Cloudinary uses to reference the image. Then I call cloudinary.uploader.destroy() with that public ID. I wrap this in a try-catch so that if the Cloudinary deletion fails for some reason, the product is still deleted from the database.

🔗 **PRO TIP:** [medium](#)

9. Security

Security questions are common. Know the 'why' behind every decision.

[MEDIUM] What security measures did you implement?

✓SAY THIS:

Several. Passwords are hashed with bcrypt and never stored as plain text. Tokens are stored in HttpOnly cookies, which are inaccessible to JavaScript, preventing XSS attacks. The sameSite: strict cookie attribute prevents CSRF attacks. The secure flag ensures cookies are only sent over HTTPS in production. Role-based access control is enforced by the adminRoute middleware on all admin endpoints, and mirrored by route guards in the React frontend. The Stripe session ID has a unique index to prevent duplicate orders. Redis token validation allows immediate logout by revoking refresh tokens.

🔗 **PRO TIP:** *Listing these in a structured way shows you thought about security systematically, not accidentally.*

[MEDIUM] What is an XSS attack and how does your app prevent it?

✓SAY THIS:

XSS — Cross-Site Scripting — is when an attacker injects malicious JavaScript into a web page. If tokens are in localStorage, that script can steal them. I prevent this by storing tokens in HttpOnly cookies. The HttpOnly flag instructs the browser to never expose the cookie to JavaScript, so even if malicious script runs on the page, it cannot access the authentication tokens.

🔗 **PRO TIP:** *medium*

[MEDIUM] What is a CSRF attack and how does your app prevent it?

✓SAY THIS:

CSRF — Cross-Site Request Forgery — is when a malicious website tricks a logged-in user's browser into making a request to your site. Because browsers automatically send cookies with requests, the request would carry the authentication cookie and appear legitimate. I set sameSite: strict on my cookies, which tells the browser to only send the cookie when the request originates from the same site. A request from a malicious third-party site will not include the cookie and will be rejected as unauthenticated.

🔗 **PRO TIP:** *hard*

[MEDIUM] What is the role of the adminRoute middleware?

✓SAY THIS:

adminRoute is a middleware function that runs after protectRoute. protectRoute verifies the JWT and attaches the user object to the request. adminRoute then checks if req.user.role equals 'admin'. If not, it returns a 403 Forbidden response immediately. This ensures that even if someone knows the admin API endpoint URLs, they cannot call them without an account that has the admin role in the database. The React frontend also hides admin routes, but the real security is always on the server.

🔗 **PRO TIP:** *The last sentence — 'real security is always on the server' — shows maturity. Say it.*

10. Deployment & DevOps

[MEDIUM] How is your application deployed on Vercel?

✓✓SAY THIS:

The build command runs `npm install` for the backend, then installs and builds the frontend with Vite, producing a `dist` folder. The `vercel.json` file configures two rewrites. Any request to `/api/*` is routed to `api/index.js`, which is a serverless function that imports and runs the Express app. Any other request is served from `frontend/dist/index.html`, which is the React SPA. This means both the frontend and backend are deployed from a single repository.

🔗 PRO TIP: *medium*

[MEDIUM] What is a serverless function?

✓✓SAY THIS:

A serverless function is a piece of backend code that runs on demand without you managing a dedicated server. When a request comes in to `/api`, Vercel spins up a container, runs the function, returns the response, and shuts down. You are only charged for execution time. The trade-off is that serverless functions are stateless — any data stored in memory or on disk is lost between invocations. This is why I use external services: MongoDB Atlas for data, Redis for sessions, and Cloudinary for images — nothing is stored on the function itself.

🔗 PRO TIP: *hard*

[MEDIUM] What are environment variables and why are they important?

✓✓SAY THIS:

Environment variables are configuration values stored outside the codebase — things like database connection strings, API keys, and JWT secrets. They are important for two reasons. First, security: if I hardcoded my MongoDB password or Stripe secret key in the source code and pushed it to GitHub, anyone could see it and access my database or steal money through Stripe. Second, flexibility: the same code can connect to a test database locally and the production database on Vercel by changing the environment variables without changing any code.

🔗 PRO TIP: *easy*

11. Tough Questions — Professor Trying to Trip You Up

These are adversarial questions. Stay calm, be honest, and show you understand trade-offs.

[MEDIUM] What are the limitations or weaknesses of your project?

✓SAY THIS:

A few honest limitations. There is no product search or filtering — users can only browse by category. There are no automated tests, so regressions could go unnoticed. Email notifications are not implemented, so customers do not receive confirmation emails. The Redis cache for featured products does not have a TTL fallback, so if Redis goes down temporarily, the cache could become permanently stale until the next admin action. These are areas I would improve given more time.

🔗 **PRO TIP:** *Acknowledging limitations honestly is far better than being caught out. It shows engineering maturity.*

[MEDIUM] What happens if MongoDB goes down?

✓SAY THIS:

All database operations would fail. API endpoints that query MongoDB would throw errors. The Express error handling middleware catches unhandled errors and returns a 500 Internal Server Error response. The frontend would show error states. The Redis cache would still serve featured products, so the homepage would remain functional temporarily. To handle this properly in production I would implement retry logic and circuit breakers, but for this project the error handling is basic.

🔗 **PRO TIP:** *hard*

[MEDIUM] What is the difference between authentication and authorisation?

✓SAY THIS:

Authentication is verifying who you are — proving your identity. In my project, login is authentication: you provide credentials and the server confirms you are a real registered user and issues a token. Authorisation is verifying what you are allowed to do. The adminRoute middleware is authorisation: it checks whether the authenticated user has the admin role before allowing access to admin endpoints. You can be authenticated without being authorised — a logged-in customer is authenticated but not authorised to access the admin dashboard.

🔗 **PRO TIP:** *medium*

[MEDIUM] Can a customer access admin APIs if they know the URL?

✓SAY THIS:

No. The adminRoute middleware on the server checks `req.user.role === 'admin'`. Even if a customer sends a request directly to `/api/orders/admin/all` using Postman or curl, the middleware will return 403 Forbidden because their user document in MongoDB has role: 'customer'. The frontend hiding the dashboard is a UX convenience, not a security measure — the real protection is always the server-side middleware.

🔗 **PRO TIP:** *hard*

[MEDIUM] Why does the cart store data in the database instead of just localStorage?

✓SAY THIS:

Storing the cart in MongoDB means it persists across devices and sessions. If a user adds items on their phone, logs in on their laptop, and the cart is in localStorage, it would be empty on the laptop. By storing cartItems in the User document in MongoDB and loading it on login via `getCartItems()`, the cart follows the user wherever they log in. localStorage is also wiped when the user clears browser data.

🔗 **PRO TIP:** *medium*

[MEDIUM] What is CORS and why did you configure it?

✓SAY THIS:

CORS — Cross-Origin Resource Sharing — is a browser security mechanism that blocks web pages from making requests to a different domain than the one that served the page. My React frontend runs on a different origin than my Express backend during development. Without CORS configuration, the browser would block all API

calls. I configured the cors middleware in Express to explicitly allow requests from my frontend URL and to allow credentials, which is required for cookies to be sent cross-origin.

 **PRO TIP:** *medium*

12. Quick-Fire Definitions — 30 Second Answers

These may be asked rapidly one after another. Keep answers to 2–3 sentences maximum.

Term	What to Say
Middleware	A function that runs between the request and the route handler. It can modify req/res, run logic, or end the request early. Examples: protectRoute, cors, cookieParser.
Mongoose populate()	Replaces a stored ObjectId reference with the actual document from the referenced collection. Like a SQL JOIN but done in application code.
Async / Await	Syntax for handling asynchronous operations — things that take time like database queries — without callback hell. Await pauses execution until the Promise resolves.
HTTP status codes	200 OK, 201 Created, 400 Bad Request (client error), 401 Unauthorised, 403 Forbidden, 404 Not Found, 500 Internal Server Error.
CDN	Content Delivery Network. A network of servers distributed globally that serve static assets like images from the server geographically closest to the user, reducing latency. Cloudinary uses a CDN.
Base64	A way to encode binary data (like an image file) as a text string. Used to send images in JSON request bodies since JSON is text-based.
Idempotent	An operation that produces the same result no matter how many times you call it. My checkout-success endpoint is idempotent — calling it twice with the same session ID does not create two orders.
Stateless	The server does not store any session information between requests. Each request must contain all the information needed to process it. REST APIs and JWTs are stateless.
TTL	Time To Live. A duration after which data automatically expires and is deleted. I set a 7-day TTL on refresh tokens in Redis.
Lean query (Mongoose)	Calling .lean() on a Mongoose query returns a plain JavaScript object instead of a full Mongoose document. It is faster because it skips the overhead of Mongoose document methods — useful for read-only operations.

13. Day-Of Cheat Sheet

Read this page the morning of your viva.

The 5 Things Most Likely to Be Asked About

- JWT refresh token rotation + why Redis is needed for logout
- The Stripe payment flow end-to-end (create session → metadata → success endpoint)
- Why price is stored in the order, not just a product reference
- HttpOnly cookies vs localStorage (XSS prevention)
- The order statusHistory — what it is and why it matters

3 Phrases That Impress Professors

- 'I made this decision to preserve data integrity' — use when explaining price snapshots in orders.
- 'The real security is always server-side' — use when explaining admin route protection.
- 'I used Redis here because it has native TTL support and sub-millisecond reads' — for any Redis question.

If You Don't Know the Answer

- Do NOT say 'I don't know' and go silent.
- Say: 'I haven't gone deep into that specific topic, but based on what I know, I would expect it works like...'
- Or: 'In my implementation I handled it this way — [explain what you actually did]'
- Professors give credit for reasoning, not just memorised answers.

The Golden Rule

You built this project. Every line of code exists because you or the architecture required it. Answer from first principles, not from memorisation. If you understand WHY each decision was made, you can answer any variation of the question.